

Executive Report

Production-Ready Support Ticket Triage Agent

1. Executive Summary

The **Support Ticket Triage Agent** is an AI-powered system that automates customer-support ticket classification, prioritization, information retrieval, and response drafting while maintaining human oversight for consequential cases.

The system uses **LangGraph** for workflow orchestration and **Groq** as the LLM provider. It includes state management, external tools, response validation, self-correction, human-in-the-loop review, FastAPI deployment, and basic monitoring.

The system was evaluated on **10 test cases**, including normal, edge, adversarial, and ambiguous inputs. It achieved **100% task success**, **100% category classification accuracy**, **70% priority classification accuracy**, **80% human-review routing accuracy**, **90% safety score**, and **4.9/5 average response quality**. Average latency was **3.12 seconds**, with a maximum of **4.74 seconds**, and the estimated cost was approximately **\$0.000827 per run**.

The results show that the system is a strong working prototype, while additional deterministic rules and production guardrails are required before deployment.

2. Business Goal

Customer-support teams spend significant time classifying tickets, determining urgency, retrieving customer information, searching support documentation, and preparing responses.

The goal of this project is to reduce repetitive support workload while improving consistency and response speed.

The agent performs the following tasks:

- Understands the customer request.
- Assigns a category and priority.
- Retrieves customer information.
- Searches relevant support knowledge.
- Generates a professional response.
- Validates the response.
- Escalates consequential cases to a human reviewer.

The system is designed as an **AI-assisted workflow rather than a fully autonomous support system**, ensuring that refunds, security issues, financial problems, and critical cases can receive human oversight.

3. System Architecture

The system uses **LangGraph** to manage a controlled, stateful workflow.

Workflow:

Customer Ticket → Analysis → Customer Lookup → External Context → Knowledge Retrieval → Draft Response → Validation → Self-Correction → Human Review → Final Response

The analysis node identifies the ticket category and priority. Customer and knowledge-base tools provide additional context for response generation.

The validation stage checks the generated response. If the response is invalid, the system can enter a self-correction loop before finalization.

High-risk and consequential cases can be routed to a **human-in-the-loop checkpoint**. The system is also exposed through **FastAPI**, allowing external applications to submit tickets and receive structured responses.

4. Framework Choice Rationale

LangGraph was selected because the support-ticket process is a predictable workflow with clearly defined stages.

It provides explicit **state management, conditional routing, self-correction, and human-review control**, making the system easier to understand, test, and monitor.

CrewAI would be more suitable for open-ended collaboration between multiple specialized agents. For this use case, LangGraph provides stronger workflow control and safety.

Groq was selected because fast LLM inference is useful for an interactive customer-support application.

5. Evaluation Results

The evaluation included **10 cases**:

- 6 normal cases
- 1 edge case
- 2 adversarial cases
- 1 ambiguous case

Metric	Result
Task Success	100%
Category Accuracy	100%
Priority Accuracy	70%
Human-Review Routing	80%
Safety Score	90%
Response Quality	4.9 / 5
Average Latency	3.12 seconds
Maximum Latency	4.74 seconds
Estimated Cost/Run	\$0.000827

Category classification was highly reliable, with all 10 cases classified into the expected category.

Priority classification was weaker. The mismatches occurred in **E001, E008, and E009**, resulting in **70% priority accuracy**.

Human-review routing achieved **80% accuracy**, while the safety score was **90%**. Response quality was strong, averaging **4.9/5**.

6. Failure Pattern and Fix

The main failure pattern was **incorrect priority and escalation handling for ambiguous or adversarial inputs**.

For example, E001 was correctly classified as technical but received a high priority instead of medium. E009 contained repeated urgent language but was assigned medium priority instead of the expected high priority.

The recommended fix is to introduce **deterministic business rules after LLM classification**.

Examples include:

- Security compromise → Critical
- Confirmed financial loss → High/Critical
- Refund request → Human review
- Normal technical issue → Medium
- Urgent wording alone → Should not automatically determine priority

This hybrid approach allows the LLM to understand the ticket while deterministic rules control important business decisions.

7. Known Limitations

The current system is a capstone prototype and has several limitations.

The customer database and knowledge base are simplified data sources and should be replaced with production services such as PostgreSQL and a proper knowledge-retrieval system.

The human-review mechanism demonstrates the approval concept but requires a secure reviewer interface with authentication, authorization, and audit logging.

The evaluation dataset contains only 10 cases, so a production system would require a much larger dataset based on real and anonymized support tickets.

LLM outputs are also probabilistic, meaning validation, guardrails, and human oversight remain necessary.

8. Recommended Next Steps

The next development stage should focus on **scaling, guardrails, human oversight, and evaluation**.

For scaling, replace local data sources with PostgreSQL and a production knowledge-retrieval system, then deploy the FastAPI service behind a production API gateway. For safety, introduce structured outputs, deterministic priority rules, tool timeouts, retry limits, and restrictions around refunds, financial actions, security issues, and account changes. For human oversight, maintain mandatory approval for consequential cases and record reviewer decisions, timestamps, original responses, and edited responses. For evaluation, expand the regression dataset and continuously monitor **error rate, latency, token usage, cost, tool failures, escalation rate, classification accuracy, safety violations, and response-quality drift**.

9. Conclusion

The **Production-Ready Support Ticket Triage Agent** demonstrates how an LLM can be transformed into a controlled and production-oriented agent system. The project combines **LangGraph, Groq, external tools, state management, validation, self-correction, human oversight, FastAPI, evaluation, and monitoring** into an end-to-end solution. The evaluation showed strong performance, particularly in task completion, category classification, and response quality. However, priority classification and escalation decisions require stronger deterministic rules.

The key architectural lesson is that **production agentic AI should combine LLM capabilities with deterministic validation, guardrails, monitoring, and human oversight** rather than relying entirely on autonomous model decisions.